

## GUIDE D'INSTALLATION

# Installation de Whisper

Via faster-whisper — Windows

## AUTEURE

E. Ryckewaert

Univ. Lille, CNRS, UMR 9193 — SCALab

Sciences Cognitives et Sciences Affectives

F-59000 Lille, France

## À RETENIR AVANT TOUT

Tout se passe **sur votre ordinateur** : ni les enregistrements ni les transcriptions ne sont envoyés sur un serveur. L'installation se fait une seule fois ; ensuite, transcrire un dossier d'entretiens tient en une commande.

## Sommaire

- |          |                                           |           |                                   |
|----------|-------------------------------------------|-----------|-----------------------------------|
| <b>1</b> | Pourquoi cette méthode                    | <b>6</b>  | Comprendre les réglages du script |
| <b>2</b> | Prérequis                                 | <b>7</b>  | Option : identifier qui parle     |
| <b>3</b> | Installation en cinq étapes               | <b>8</b>  | Confidentialité des données       |
| <b>4</b> | Ce que produit le script                  | <b>9</b>  | Limites connues                   |
| <b>5</b> | Le script à copier — <i>transcrire.py</i> | <b>10</b> | Dépannage                         |

## 1 Pourquoi cette méthode

Cette méthode utilise directement Python et pip, sans passer par Anaconda, plus lourd et sujet à des difficultés de téléchargement selon la puissance de l'ordinateur. Elle est plus simple à mettre en place pour un usage ponctuel de transcription, et fonctionne **sans carte graphique dédiée** : le traitement se fait sur le processeur.

| TEMPS D'INSTALLATION                                        | ENSUITE                               | CE QUE VOUS OBTENEZ                                             |
|-------------------------------------------------------------|---------------------------------------|-----------------------------------------------------------------|
| ≈ 30 min la première fois, dont le téléchargement du modèle | Un clic par dossier d'enregistrements | Trois fichiers par enregistrement : texte, texte horodaté, Word |

## 2 Prérequis

### 2.1 — Python (version 3.9 à 3.13)

À télécharger depuis [python.org](https://python.org).

#### LA CASE À NE PAS MANQUER

Pendant l'installation, cochez « **Add python.exe to PATH** ».

### 2.2 — ffmpeg

Nécessaire pour lire les fichiers audio. Dans l'invite de commandes copier le texte suivant :

```
winget install ffmpeg
```

Vérifier ensuite que tout répond en copiant : `python --version` puis `ffmpeg -version`

## 3 Installation en cinq étapes

**01**

### Créer un dossier de travail

Par exemple nommé "whisper", et y placer tous vos fichiers audio à transcrire.

**02**

### Ouvrir une invite de commandes dans ce dossier

Tapez cmd dans la barre d'adresse de l'Explorateur de fichiers. Si vous n'y parvenez pas copier directement dans l'invite de commande : pushd "chemin\du\dossier"

**03**

### Installer les dépendances, à copier dans l'invite de commandes :

```
python -m pip install faster-whisper python-docx tqdm
```

**04**

### Créer un fichier texte dans ce même dossier : transcrire.py

Dans ce fichier texte copier coller à l'identique le code fourni plus bas.

**05**

### Lancer la transcription en copiant dans l'invite de commandes :

```
python transcrire.py
```

Pour arrêter en cours de route : Ctrl + C dans l'invite de commandes.

## 4 Ce que produit le script

Le script scanne le dossier de travail et transcrit **tous** les fichiers audio qu'il y trouve (.mp3 .wav .m4a .mp4 .flac). Pour chacun, il génère trois fichiers nommés d'après l'original :

| FICHIER GÉNÉRÉ                         | CONTENU                                                                                        |
|----------------------------------------|------------------------------------------------------------------------------------------------|
| NomFichier_transcription.txt           | Texte brut, sans horodatage.                                                                   |
| NomFichier_transcription_horodatee.txt | Texte avec marqueurs temporels [00:03 - 00:12], pour retrouver un passage précis dans l'audio. |
| NomFichier_transcription.docx          | Version Word, prête à intégrer dans un document de travail.                                    |

Une barre de progression s'affiche pendant le traitement ; deux bips signalent la fin de l'ensemble du dossier selon une idée du Dr Louise Zanni.

## 5

# Le script à copier

transcrire.py

```
import os
import glob
from datetime import timedelta
from faster_whisper import WhisperModel
from tqdm import tqdm
import winsound
from docx import Document

MODELE = "large-v3" # plus precis que "medium", mais plus lent sur CPU
DOSSIER = "." # dossier courant : le script traite le dossier ou il est lance
EXTENSIONS = ("*.mp3", "*.wav", "*.m4a", "*.mp4", "*.flac")

def format_temps(secondes):
    td = timedelta(seconds=secondes)
    total_sec = int(td.total_seconds())
    h, reste = divmod(total_sec, 3600)
    m, s = divmod(reste, 60)
    if h > 0:
        return f"{h:02d}:{m:02d}:{s:02d}"
    return f"{m:02d}:{s:02d}"

def transcrire_fichier(model, chemin_audio):
    nom_base = os.path.splitext(os.path.basename(chemin_audio))[0]
    print(f"\n=== Transcription de : {chemin_audio} ===")

    segments, info = model.transcribe(
        chemin_audio,
        language="fr",
        vad_filter=True,
        vad_parameters=dict(min_silence_duration_ms=500),
        no_speech_threshold=0.4,
        compression_ratio_threshold=2.8,
        condition_on_previous_text=False,
        beam_size=5,
    )

    print(f"Langue detectee : {info.language} (probabilite {info.language_probability:.2f})")
    print(f"Duree totale : {info.duration:.1f}s")

    texte_simple = ""
    texte_horodate = ""

    try:
        pbar = tqdm(total=info.duration, unit="s", desc="Transcription",
                    bar_format="{l_bar}{bar}| {n:.0f}/{total:.0f}s", ncols=80)
    except Exception:
        pbar = None

    for segment in segments:
        texte_simple += segment.text
        debut = format_temps(segment.start)
        fin = format_temps(segment.end)
        texte_horodate += f"[{debut} - {fin}] {segment.text.strip()}\n"
        if pbar is not None:
            try:
                pbar.update(segment.end - pbar.n)
            except Exception:
                pbar = None

    if pbar is not None:
        try:
```

```
pbar.close()
except Exception:
    pass

fichier_simple = f"{nom_base}_transcription.txt"
fichier_horodate = f"{nom_base}_transcription_horodate.txt"
fichier_docx = f"{nom_base}_transcription.docx"

with open(fichier_simple, "w", encoding="utf-8") as f:
    f.write(texte_simple.strip())

with open(fichier_horodate, "w", encoding="utf-8") as f:
    f.write(texte_horodate.strip())

doc = Document()
doc.add_heading(nom_base, level=1)
doc.add_paragraph(texte_simple.strip())
doc.save(fichier_docx)

print(f"-> {fichier_simple}")
print(f"-> {fichier_horodate}")
print(f"-> {fichier_docx}")

def main():
    model = WhisperModel(MODELE, device="cpu", compute_type="int8")

    fichiers_audio = []
    for ext in EXTENSIONS:
        fichiers_audio.extend(glob.glob(os.path.join(DOSSIER, ext)))

    if not fichiers_audio:
        print("Aucun fichier audio trouve dans ce dossier.")
        return

    print(f"{len(fichiers_audio)} fichier(s) audio trouve(s) :")
    for f in fichiers_audio:
        print(f" - {f}")

    for chemin_audio in fichiers_audio:
        transcrire_fichier(model, chemin_audio)

    print("\nToutes les transcriptions sont terminees.")
    winsound.Beep(1000, 200)
    winsound.Beep(1200, 200)

if __name__ == "__main__":
    main()
```

Enregistrer sous transcrire.py dans le dossier de travail.

## 6 Comprendre les réglages

Rien n'est à modifier pour que le script fonctionne. Ce tableau est présent pour vous aider à comprendre les lignes et savoir *quoi* changer au besoin.

| LIGNE                                                                                                       | À QUOI ELLE SERT                                                                                                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>MODELE = "large-v3"</code>                                                                            | Le modèle utilisé. Plus il est grand ( <i>tiny</i> → <i>small</i> → <i>medium</i> → <i>large-v3</i> ), plus il est précis et lent sur processeur. <b>Passez à "medium" pour aller plus vite, au prix d'une légère perte de précision.</b> |
| <code>DOSSIER = "."</code>                                                                                  | Le point signifie « dossier courant » : le script traite le dossier depuis lequel il est lancé.                                                                                                                                           |
| <code>glob.glob(...)</code>                                                                                 | Constitue automatiquement la liste des fichiers audio du dossier : aucune saisie manuelle de noms n'est nécessaire.                                                                                                                       |
| <code>vad_parameters</code><br><code>no_speech_threshold</code><br><code>compression_ratio_threshold</code> | Réduisent les mots omis ou « corrigés » à tort — par exemple « j'y pensais pas » transformé en « je n'y pensais pas ». Ils limitent la tendance du modèle à lisser l'oral relâché vers un français écrit.                                 |
| <code>condition_on_previous_text=False</code>                                                               | Empêche le modèle de « deviner » la suite à partir du texte déjà transcrit, ce qui réduit les reformulations automatiques.                                                                                                                |
| <code>try / except</code> autour de <code>tqdm</code>                                                       | Protège contre un bug connu de la barre de progression sur certains terminaux Windows, qui peut sinon interrompre le script juste avant l'enregistrement des fichiers.                                                                    |
| <code>doc.add_paragraph(...)</code><br><code>doc.save(...)</code>                                           | Génèrent la version Word du texte simple.                                                                                                                                                                                                 |
| <code>winsound.Beep(1000, 200)</code>                                                                       | Émet le signal sonore de fin, une fois tous les fichiers traités.                                                                                                                                                                         |

## 7

## Option : identifier qui parle

Diarisation

### CECI EST UNE OPTION, PAS UNE ÉTAPE OBLIGATOIRE

Le script du 5) suffit pour obtenir une transcription complète. Cette section n'est utile que si vous avez besoin de savoir **qui dit quoi**, par exemple distinguer deux ou trois personnes qui parlent dans un même entretien.

La mise en place est plus longue que le script de base, car elle repose sur des modèles tiers (pyannote.audio) dont l'accès est contrôlé individuellement. Concrètement, **chaque personne** doit créer son propre compte gratuit **Hugging Face**, accepter les conditions de trois modèles, et générer son propre jeton d'accès. Rien de tout cela n'est transférable d'une personne à une autre.

Cette mise en place ne se fait **qu'une seule fois par personne**. Ensuite, l'usage quotidien est aussi simple que le script de base.

### 7.1 — Mise en place, une seule fois

#### 01 Créer un compte gratuit

Sur [huggingface.co](https://huggingface.co).

#### 02 Accepter les conditions de trois modèles

Bouton « Agree and access repository » sur chacune de ces pages :

[huggingface.co/pyannote/speaker-diarization-3.1](https://huggingface.co/pyannote/speaker-diarization-3.1)

[huggingface.co/pyannote/segmentation-3.0](https://huggingface.co/pyannote/segmentation-3.0)

[huggingface.co/pyannote/speaker-diarization-community-1](https://huggingface.co/pyannote/speaker-diarization-community-1)

#### 03 Générer un jeton d'accès (Token)

Sur [huggingface.co/settings/tokens](https://huggingface.co/settings/tokens), de type **Read**.

#### 04 Installer la bibliothèque supplémentaire

```
python -m pip install pyannote.audio
```

#### 05 Enregistrer le jeton, sans l'écrire dans un fichier

```
setx HF_TOKEN "coller_le_token_ici_par_clic_droit"
```

Puis **fermer la fenêtre et en ouvrir une nouvelle** : setx ne prend effet que dans les fenêtres ouvertes après la commande. Vérifier avec `echo %HF_TOKEN%`.

**NE PAS FAIRE PASSER LE JETON PAR WORD**

Un copier-coller via un traitement de texte comme Word peut corrompre silencieusement certains caractères : des lettres visuellement identiques mais en réalité différentes s'y substituent. Passez par le Bloc-notes, qui ne modifie rien, et collez le jeton à partir de là.

## 7.2 — Le script avec locuteurs

À enregistrer sous `transcrire_locuteurs.py`. Il reprend l'intégralité du script de base — mêmes réglages, même barre de progression, mêmes trois sorties — et ajoute une quatrième sortie avec les locuteurs. **Rien à modifier** : le jeton est lu automatiquement depuis la variable d'environnement définie plus haut.

```
import os
import glob
from datetime import timedelta
import numpy as np
import torch
import av
from faster_whisper import WhisperModel
from pyannote.audio import Pipeline
from tqdm import tqdm
from docx import Document
import winsound

MODELE_WHISPER = "large-v3"
DOSSIER = "." # dossier courant
EXTENSIONS = ("*.mp3", "*.wav", "*.m4a", "*.mp4", "*.flac")

# Le token est lu depuis la variable d'environnement HF_TOKEN
# (définie une fois avec la commande setx), jamais stocke dans ce fichier.
HF_TOKEN = os.environ.get("HF_TOKEN")

# Nombre de personnes qui parlent dans l'audio.
# Mets 2 si c'est toujours un entretien a deux (aidant + personne malade),
# ou laisse None pour que le programme le devine automatiquement (moins fiable).
NOMBRE_LOCUTEURS = 2
FREQUENCE_CIBLE = 16000 # frequence attendue par le modele de diarisation

def format_temps(secondes):
    td = timedelta(seconds=secondes)
    total_sec = int(td.total_seconds())
    h, reste = divmod(total_sec, 3600)
    m, s = divmod(reste, 60)
    if h > 0:
        return f"{h:02d}:{m:02d}:{s:02d}"
    return f"{m:02d}:{s:02d}"

def charger_audio_pour_diarisation(chemin_audio, frequence_cible=FREQUENCE_CIBLE):
    """
    Decode le fichier audio nous-memes avec la bibliotheque 'av'
    (deja installee via faster-whisper), pour eviter de dependre
    de torchcodec / FFmpeg partage qui pose probleme sous Windows.
    """
    conteneur = av.open(chemin_audio)
    flux_audio = conteneur.streams.audio[0]
    reechantillonneur = av.audio.resampler.AudioResampler(
        format="s16", layout="mono", rate=frequence_cible
    )
```

```

morceaux = []
for frame in conteneur.decode(flux_audio):
    frame.pts = None
    for frame_reechantillonnee in reechantillonneur.resample(frame):
        morceaux.append(frame_reechantillonnee.to_ndarray())
conteneur.close()

audio_np = np.concatenate(morceaux, axis=1).astype(np.float32) / 32768.0
waveform = torch.from_numpy(audio_np)
return waveform, frequence_cible

def extraire_annotation(resultat_diarisation):
    """
    Les versions recentes de pyannote.audio renvoient un objet qui encapsule
    le resultat plutot que de le renvoyer directement. Cette fonction
    retrouve l'annotation utilisable quel que soit le format renvoye.
    """
    if hasattr(resultat_diarisation, "itertracks"):
        return resultat_diarisation
    for attribut in ("speaker_diarization", "annotation", "diarization"):
        if hasattr(resultat_diarisation, attribut):
            return getattr(resultat_diarisation, attribut)
    print("ATTENTION : format inconnu. Attributs disponibles :", dir(resultat_diarisation))
    raise AttributeError("Impossible de trouver l'annotation dans le resultat de la diarisation.")

def assigner_locuteur(debut, fin, annotation):
    meilleur_locuteur = None
    meilleur_overlap = 0
    for turn, _, locuteur in annotation.itertracks(yield_label=True):
        overlap = min(fin, turn.end) - max(debut, turn.start)
        if overlap > meilleur_overlap:
            meilleur_overlap = overlap
            meilleur_locuteur = locuteur
    return meilleur_locuteur or "Locuteur inconnu"

def transcrire_fichier(model_whisper, pipeline_diarisation, chemin_audio):
    nom_base = os.path.splitext(os.path.basename(chemin_audio))[0]
    print(f"\n=== Traitement de : {chemin_audio} ===")

    print("Etape 1/2 : transcription du texte...")
    segments, info = model_whisper.transcribe(
        chemin_audio,
        language="fr",
        vad_filter=True,
        vad_parameters=dict(min_silence_duration_ms=500),
        no_speech_threshold=0.4,
        compression_ratio_threshold=2.8,
        condition_on_previous_text=False,
        beam_size=5,
    )
    print(f"Langue detectee : {info.language} (probabilite {info.language_probability:.2f})")
    print(f"Duree totale : {info.duration:.1f}s")

    texte_simple = ""
    texte_horodate = ""
    segments_liste = []

    try:
        pbar = tqdm(total=info.duration, unit="s", desc="Transcription",
                    bar_format="{l_bar}{bar}| {n:.0f}/{total:.0f}s", ncols=80)
    except Exception:
        pbar = None

    for segment in segments:
        segments_liste.append(segment)

```

```

    texte_simple += segment.text
    debut = format_temps(segment.start)
    fin = format_temps(segment.end)
    texte_horodate += f"[{debut} - {fin}] {segment.text.strip()}\n"
    if pbar is not None:
        try:
            pbar.update(segment.end - pbar.n)
        except Exception:
            pbar = None

if pbar is not None:
    try:
        pbar.close()
    except Exception:
        pass

print("Etape 2/2 : identification des locuteurs (peut prendre plusieurs minutes)...")
waveform, frequence = charger_audio_pour_diarisation(chemin_audio)
resultat_diarisation = pipeline_diarisation(
    {"waveform": waveform, "sample_rate": frequence},
    num_speakers=NOMBRE_LOCUTEURS,
)
annotation = extraire_annotation(resultat_diarisation)

texte_locuteurs = ""
dernier_locuteur = None
for segment in segments_liste:
    locuteur = assigner_locuteur(segment.start, segment.end, annotation)
    if locuteur != dernier_locuteur:
        texte_locuteurs += f"\n\n[{format_temps(segment.start)}] {locuteur} :\n"
        dernier_locuteur = locuteur
    texte_locuteurs += segment.text.strip() + " "

fichier_simple = f"{nom_base}_transcription.txt"
fichier_horodate = f"{nom_base}_transcription_horodate.txt"
fichier_docx = f"{nom_base}_transcription.docx"
fichier_locuteurs = f"{nom_base}_transcription_locuteurs.txt"

with open(fichier_simple, "w", encoding="utf-8") as f:
    f.write(texte_simple.strip())

with open(fichier_horodate, "w", encoding="utf-8") as f:
    f.write(texte_horodate.strip())

with open(fichier_locuteurs, "w", encoding="utf-8") as f:
    f.write(texte_locuteurs.strip())

doc = Document()
doc.add_heading(nom_base, level=1)

# Le Word integre directement les tours de parole (locuteur + horodatage),
# plutot que le texte brut sans distinction des interlocuteurs.
blocs = [b.strip() for b in texte_locuteurs.strip().split("\n\n") if b.strip()]
for bloc in blocs:
    lignes = bloc.split("\n", 1)
    entete = lignes[0].strip()
    texte_bloc = lignes[1].strip() if len(lignes) > 1 else ""

    p_entete = doc.add_paragraph()
    run_entete = p_entete.add_run(entete)
    run_entete.bold = True

    if texte_bloc:
        doc.add_paragraph(texte_bloc)

```

```
doc.save(fichier_docx)

print(f"-> {fichier_simple}")
print(f"-> {fichier_horodate}")
print(f"-> {fichier_docx}")
print(f"-> {fichier_locuteurs}")

def main():
    if not HF_TOKEN:
        print("ERREUR : la variable d'environnement HF_TOKEN n'est pas definie.")
        print('Rappel : setx HF_TOKEN "ton_token" dans une invite de commandes, puis ouvrir une NOUVELLE fenetre.')
        return

    print("Chargement des modeles (transcription + diarisation)...")
    model_whisper = WhisperModel(MODELE_WHISPER, device="cpu", compute_type="int8")
    pipeline_diarisation = Pipeline.from_pretrained(
        "pyannote/speaker-diarization-3.1",
        token=HF_TOKEN,
    )

    fichiers_audio = []
    for ext in EXTENSIONS:
        fichiers_audio.extend(glob.glob(os.path.join(DOSSIER, ext)))

    if not fichiers_audio:
        print("Aucun fichier audio trouve dans ce dossier.")
        return

    print(f"{len(fichiers_audio)} fichier(s) audio trouve(s) :")
    for f in fichiers_audio:
        print(f" - {f}")

    for chemin_audio in fichiers_audio:
        transcrire_fichier(model_whisper, pipeline_diarisation, chemin_audio)

    print("\nTous les fichiers ont ete traites.")
    winsound.Beep(1000, 200)
    winsound.Beep(1200, 200)

if __name__ == "__main__":
    main()
```

Lancement : `python transcrire_locuteurs.py`

## 7.3 — Ce que ce script ajoute

| LIGNE                                    | POURQUOI ELLE EST LÀ                                                                                                                                                                                                 |
|------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HF_TOKEN =<br>os.environ.get("HF_TOKEN") | Lit le jeton depuis la variable d'environnement Windows plutôt que depuis le fichier, pas de risque de corruption, ni de partage accidentel (dépôt GitHub public, envoi du script par mail).                         |
| charger_audio_pour_diarisation(...)      | Décode l'audio avec la bibliothèque av plutôt que de laisser pyannote le faire. Contourne un bug fréquent sous Windows (message « Could not load libtorchcodec »).                                                   |
| extraire_annotation(...)                 | Les versions de pyannote.audio ont changé la structure de leur résultat. Cette fonction retrouve l'information utile quel que soit le format, et affiche un message clair si un format nouveau apparaît.             |
| NOMBRE_LOCUTEURS = 2                     | À ajuster selon le nombre réel de personnes qui parlent. Mettez None si ce nombre varie d'un fichier à l'autre, détection automatique mais moins fiable.                                                             |
| blocs = [...]<br>run_entete.bold = True  | Écrivent le Word <b>tour de parole par tour de parole</b> , locuteur et horodatage en gras, puis le texte, au lieu d'un bloc de texte brut sans distinction des interlocuteurs. C'est la version à relire dans Word. |

## 7.4 — La quatrième sortie

En plus des trois fichiers habituels, un fichier `NomFichier_transcription_locuteurs.txt` est généré, organisé par tour de parole :

```
[00:00] SPEAKER_00 :
Bonjour, merci d'être venus...

[00:14] SPEAKER_01 :
Je vous en prie...
```

### DEUX CHOSES À SAVOIR

Les libellés (SPEAKER\_00, SPEAKER\_01...) sont génériques : le programme distingue des voix différentes mais ne sait pas qui est qui. Identifiez vous-même, au début de chaque fichier, quel libellé correspond à quelle personne — **l'ordre peut changer d'un fichier à l'autre**.

Le traitement est nettement plus lent qu'une transcription simple : la diarisation ajoute une analyse complète de l'audio. Sans carte graphique dédiée, comptez bien plus que la durée réelle de l'enregistrement.

---

## 8 Confidentialité des données

Les deux scripts traitent l'audio **entièrement en local**, sur l'ordinateur qui les exécute. Ni les fichiers audio, ni le texte transcrit ne sont envoyés à un serveur externe pendant le traitement — contrairement aux outils de transcription en ligne (Otter.ai, API OpenAI, etc.).

Seule exception : au tout premier lancement, les modèles utilisés (Whisper, et pyannote pour l'option diarisation) sont téléchargés depuis Hugging Face. Ce téléchargement ne concerne que les fichiers génériques du modèle — jamais vos données.

---

### À VÉRIFIER AVANT USAGE SUR DES DONNÉES DE RECHERCHE

Que le consentement des participants couvre l'usage d'un outil de transcription automatique

Que les fichiers audio et les transcriptions restent à l'emplacement validé par le laboratoire.

Que toute information identifiante mentionnée à l'oral, noms, lieux, soit anonymisée manuellement après transcription : **l'outil ne le fait pas**

---

## 9 Limites connues

**01** Le premier lancement télécharge le modèle choisi, jusqu'à environ **3 Go** pour *large-v3*, davantage avec la diarisation. Une connexion stable est nécessaire. Les lancements suivants réutilisent les modèles déjà téléchargés.

**02** La diarisation nécessite un compte Hugging Face individuel ; elle ne peut pas être rendue disponible sans cette étape, pour des raisons de licence des modèles.

**03** Le traitement sur processeur est lent par nature : prévoyez de lancer un gros dossier en fin de journée.

---

## 10 Dépannage

| MESSAGE OU SYMPTÔME                                          | QUE FAIRE                                                                                                                                                                    |
|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 'python' n'est pas reconnu                                   | La case « Add python.exe to PATH » n'a pas été cochée (§2.1). Réinstallez Python en la cochant.                                                                              |
| Aucun fichier audio trouve dans ce dossier                   | L'invite de commandes n'est pas ouverte dans le bon dossier, ou les extensions ne figurent pas dans la liste. Vérifiez avec <code>dir</code> .                               |
| Could not load libtorchcodec                                 | Bug de chargement audio de pyannote sous Windows. Le script du §7.2 le contourne déjà : vérifiez que vous utilisez bien cette version.                                       |
| HF_TOKEN n'est pas définie                                   | La commande <code>setx</code> a été passée, mais la fenêtre n'a pas été rouverte. Fermez-la, ouvrez-en une nouvelle, vérifiez avec <code>echo %HF_TOKEN%</code> .            |
| Échec d'authentification Hugging Face malgré un jeton valide | Le jeton a probablement été abîmé par un passage dans un traitement de texte comme Word. Régénérez-le et collez-le depuis le Bloc-notes directement dans <code>setx</code> . |
| Accès refusé à un modèle pyannote                            | Les conditions des <b>trois</b> pages n'ont pas toutes été acceptées (§7.1, étape 02). Elles se valident une par une, avec le compte qui a généré le jeton.                  |

---

Une erreur non listée, un réglage qui a changé ? Signalez-la pour la prochaine version de ce guide.

**FIN DU GUIDE**